



Rapport de Projet

DS4H

Hugo CLAVEILLE

Master 1 Informatique

6 mai 2026

Table des matières

1	Introduction	3
1.1	Evolutions . . . vers l'infrastructure cloud	3
1.1.1	Abstraction	3
1.1.2	Accès distant	3
1.1.3	Collaboration	4
1.1.4	As a service	4
1.1.5	La vision commerciale	4
1.2	Les impacts informatiques	5
1.2.1	Le déploiement	5
1.2.2	Transparence	5
1.2.3	Infrastructure as Code : IaC	5
1.2.4	Développement-Exploitation : DEVOPS	6
1.2.5	Machines virtuelles et virtualisation	7
1.3	Différentes méthodes	9
1.3.1	Cloud Public	9
1.3.2	Cloud Privé	10
1.3.3	Cloud Hybride	11
1.4	Modèles d'exploitation	12
2	Objectif du travail	13
2.1	Qu'est ce que le cloud	13

2.1.1	Caractéristiques clés (NIST/ISO, 2026)	13
2.1.2	Évolutions récentes (2026)	14
2.1.3	Pourquoi cette définition est-elle importante pour notre projet ? . .	14
2.2	Contexte d'utilisation	15
2.3	Analyse des besoins	15
3	Choix de la techno	16
3.1	Solutions envisagées	16
3.1.1	OpenStack	16
3.1.2	Kubernetes (k3s)	16
3.1.3	Dokku	17
3.1.4	CapRover	17
3.1.5	Coolify	17
3.2	Tableau comparatif	18
3.3	Justification du choix : Dokploy	18
4	Mise en place de l'infrastructure	19
4.1	Dokploy	19
4.1.1	Installation et configuration	19
4.1.2	Services proposés par Dokploy	19
4.1.3	Avantages et inconvénients de Dokploy	20
4.2	Cas d'usage : déploiement d'un RNN de classification de texte	20
4.2.1	Déploiement avec Dokploy	20
4.2.2	Tests de scalabilité et de performance	21
4.3	Scalabilité avec Dokploy	21

1. Introduction

Je commencerais par évoquer les concepts sous-jacents à l'apparition du cloud. Il s'agit de "poser" les notions manipulées par le travail qui va suivre.

Comme souvent, les choses se construisent au fur et à mesure.

1.1 Evolutions ...vers l'infrastructure cloud

1.1.1 Abstraction

Vouloir abstraire la machine sur laquelle on veut faire un exécuter un programme est un souhait qui apparaît avec les premiers systèmes d'exploitation.

Bien avant le cloud, au milieu des années 60, Multics (Multiplexed Information and Computing Service) est un système d'exploitation conçu avec une vision de "Computer Utility" (service informatique public).

Pour ses créateurs (MIT, Bell Labs et GE), la puissance de calcul doit être "accédée" comme l'électricité : vous branchez une console dans le mur, et vous avez accès à une ressource partagée, sans savoir où se trouve la machine.

Multics avait la philosophie du Cloud (l'informatique comme service public partagé) et l'architecture du Cloud (modularité, isolation, persistance), mais il lui manquait la puissance des machines et la couche réseau mondiale pour aboutir.

1.1.2 Accès distant

Les machines ont toujours été développées avec l'idée d'être accédées à distance.

G.Stibitz développe le premier additionneur électromécanique sur 2 bits en 1937. [8]

Immédiatement, il complexifie son calculateur et imagine un accès distant. Lors d'une démonstration à la réunion de l'American Mathematical Society au Dartmouth College en septembre 1940, Stibitz utilise un télétype modifié pour envoyer des commandes par

lignes télégraphiques au Complex Number Calculator situé à New York. [5]

1.1.3 Collaboration

Dans les années 1990, l'émergence des réseaux et la multiplication des machines permettent l'émergence du Grid Computing (Informatique en Grille).

Tous ces unités centrales (PC par exemple), qui sont souvent sous utilisées, pourraient être mobilisées pour accélérer le traitement d'une tâche.

Le grid computing agit comme un système distribué pour le partage collaboratif des ressources. Celles-ci sont réparties entre différentes unités de calcul situées sur différents sites, pays et continents.

C'est là que le symbole du "nuage" commence à être utilisé dans les schémas réseau pour représenter ce qui se passe "à l'extérieur" de l'ordinateur local, dans une zone dont on ne gère pas l'infrastructure technique mais qui collabore pour résoudre des problèmes complexes.

1.1.4 As a service

En 1999, l'entreprise Salesforce lance un logiciel de gestion de la relation client (CRM) accessible via un simple navigateur Web.

C'est la naissance du SaaS ("Software as a Service"). C'est un changement de paradigme commercial majeur.

On ne possède plus le logiciel, on s'y abonne et on y accède par HTTP (Hypertext Transfer Protocol) jusque là utilisé pour transférer du HTML (HyperText Markup Language).

1.1.5 La vision commerciale

C'est sans doute l'étape la plus concrète car elle fait vraiment diverger le cloud d'une structure de type Grid Computing.

Dès le début des années 2000, Amazon construit une infrastructure massive qui lui permet d'absorber des pics de trafic de sa boutique en ligne comme par exemple à Noël.

Elle fait néanmoins le constat que ses ressources sont inutilisée 90 % de l'année.

Le Grid Computing a été pensé avec une vision scientifique de l'amélioration des performances. Amazon a une approche commerciale : Louer cette infrastructure vide à d'autres entreprises.

En 2006, Amazon lance AWS (Amazon Web Services) avec des services comme S3 (stockage) et EC2 (calcul). Pour la première fois, une startup pouvait louer la puissance d'un supercalculateur pour quelques dollars par heure.

1.2 Les impacts informatiques

1.2.1 Le déploiement

Puisque l'infrastructure d'exécution est externe, il ne suffit plus de compiler et d'exécuter, le logiciel doit être déployé sur cette infrastructure louée.

1.2.2 Transparence

Pour que l'externalisation de l'infrastructure soit transparente à l'usage, elle devra remplir de nombreux critères parmi lesquels une haute disponibilité, une faible latence.

Ce dernier point impacte la topologie de l'infrastructure qui tend vers un réseau ubiquitaire se rapprochant des utilisateurs.

Plusieurs instances d'un même logiciel vont coexister dans l'infrastructure.

1.2.3 Infrastructure as Code : IaC

Traditionnellement, l'infrastructure (serveurs, câbles, pare-feu) était du "Hardware" : quelque chose de solide qu'on touche, qu'on visse dans une baie et qu'on configure manuellement via des câbles ou des écrans.

Avec le cloud, "l'infrastructure devient du logiciel" (ou Infrastructure as Code - IaC). La création et la gestion de ses serveurs est traitée comme on traite le code d'une application.

L'administrateur système ne manipule plus des machines, mais des abstractions. Il devient un développeur de plateforme.

Voici les 4 piliers de cette mutation :

1. La définition par le texte (Fichiers de configuration)

Au lieu de cliquer sur des boutons dans une interface web pour créer un serveur, vous écrivez un fichier texte (souvent en format YAML ou JSON).

Exemple : "Je veux 3 serveurs avec 8 Go de RAM, situés à Paris, avec un accès ouvert sur le port 80."

Ce fichier est votre infrastructure.

Pour la déployer, vous l'envoyez à une API (via des outils comme Terraform, CloudFormation ou Pulumi), et le Cloud "matérialise" ces ressources pour vous.

2. Le versioning (Comme sur GitHub)

Puisque votre infrastructure est maintenant un fichier texte, vous pouvez utiliser les mêmes outils que les développeurs :

- Historique : Vous pouvez voir qui a modifié la taille d'un serveur il y a 3 jours.
- Rollback : Si une modification de votre réseau provoque une panne, vous "revenez à la version précédente" du fichier et le Cloud remet l'infrastructure dans son état antérieur.
- Branches : Vous pouvez avoir une branche "Test" et une branche "Production" de votre infrastructure.

3. La reproductibilité parfaite

C'est la fin du syndrome du "ça marche sur ma machine" ou du "le serveur de test n'est pas configuré comme la prod".

Avec l'infrastructure logicielle, vous utilisez exactement le même script pour créer votre environnement de développement, de test et de production.

Vous garantisiez que les règles de sécurité et les performances sont identiques partout, sans risque d'erreur humaine (un oubli de case à cocher, par exemple).

4. L'immuabilité (Le concept "Phoenix")

C'est le changement de mentalité le plus radical :

Avant si un serveur tombait malade, on passait des heures à le soigner, à modifier sa configuration manuellement pour le réparer.

Maintenant si un serveur a un problème ou doit être mis à jour, on ne le répare pas. On supprime l'instance et on relance le script de l'infrastructure. Le logiciel recrée un serveur tout neuf, parfaitement configuré, en quelques secondes.

Imaginez que vous deviez déployer votre application dans 10 pays différents pour réduire la latence.

Sans IaC, Il faudrait des semaines pour configurer manuellement chaque centre de données.

Avec l'infrastructure logicielle : Vous lancez votre script 10 fois avec une variable différente pour la région. En quelques minutes, votre infrastructure mondiale est opérationnelle.

1.2.4 Développement-Exploitation : DEVOPS

Le DEVOPS est une évolution de l'administration système qui adopte la philosophie agile pour ne plus être un goulot d'étranglement.

Cette démarche essaye faire converger des contraintes antagonistes :

- D'un côté, l'"agile" qui vise à accélérer les vitesses de développement et de mise en production.
- De l'autre, l'administration système qui cherche à trouver une solution stable, fonctionnelle, sécurisée, ...

L'Agile est né pour les développeurs. Son but est de dire : "Arrêtons de prévoir un projet sur 2 ans, travaillons par petits cycles pour avoir des retours rapides du client." ...et donc corriger rapidement et fréquemment le logiciel en construction.

Cette approche se focalise sur l'organisation du travail avec notamment la mise en place de réunions régulières et fréquentes (Scrum), l'établissement de priorités, ...

Or comme on l'a déjà souligné, avec une approche cloud, la mise en production c'est le déploiement. Souvent l'Agile s'arrêtait à la porte de la mise en production. Le développeur finissait sa tâche "Agilement", mais le serveur n'était pas prêt.

L'administrateur système (l'Ops) n'est pas enclin au changement car le changement apporte des pannes. Il est plutôt perçu comme un frein par les développeurs.

Le DevOps est né parce que l'Agile ne servait à rien si le déploiement sur les serveurs prenait 3 mois.

1.2.5 Machines virtuelles et virtualisation

La virtualisation est une technologie fondamentale qui a permis l'émergence du cloud computing. Elle consiste à abstraire les ressources matérielles d'un ordinateur afin de permettre l'exécution de plusieurs environnements isolés sur une même machine physique.

Pourquoi la virtualisation ?

Avant l'apparition de la virtualisation, un serveur physique était généralement dédié à une seule application. Cette approche entraînait une sous-utilisation importante des ressources matérielles (CPU, mémoire, stockage).

La virtualisation permet de résoudre ce problème en faisant coexister plusieurs machines virtuelles (VM) sur un même serveur physique.

Les principaux objectifs sont :

- **Optimisation des ressources** : meilleure utilisation du matériel
- **Isolation** : chaque VM est indépendante des autres
- **Flexibilité** : déploiement rapide de nouveaux environnements
- **Portabilité** : une VM peut être déplacée d'un serveur à un autre

Avec pour objectif d'utiliser au mieux les ressources matérielles, d'isoler les applications, de faciliter le déploiement et la portabilité.

Historique

La virtualisation ne date pas du cloud. Dès les années 1970, IBM développe VM/370, un système permettant à plusieurs utilisateurs d'exécuter simultanément des systèmes d'exploitation sur un même ordinateur central.

Dans les années 1990, avec la démocratisation des ordinateurs personnels, des solutions comme Windows/386 permettent d'exécuter plusieurs environnements DOS.

En 1995, la machine virtuelle Java (JVM) introduit une nouvelle forme de virtualisation : une abstraction logicielle permettant d'exécuter un programme indépendamment du matériel.

Ce n'est qu'au début des années 2000 que la virtualisation devient centrale avec l'essor du cloud computing.

Principe de fonctionnement

Une machine virtuelle est une simulation logicielle d'un ordinateur physique.

Chaque VM possède : un système d'exploitation invité, de la mémoire virtuelle, un processeur virtuel et des périphériques simulés.

Ces ressources sont fournies et gérées par un logiciel appelé **hyperviseur**.

Les hyperviseurs

L'hyperviseur est la couche logicielle qui permet de créer, exécuter et gérer les machines virtuelles.

On distingue deux types :

- **Type 1 (bare-metal)** : fonctionne directement sur le matériel

Comme VMware ESXi ou Xen, ces hyperviseurs sont utilisés dans les datacenters et le cloud pour leur performance et leur sécurité. Ils offrent une isolation forte entre les VM et un accès direct aux ressources matérielles, ce qui les rend adaptés aux environnements de production à grande échelle.

- **Type 2** : fonctionne au-dessus d'un système d'exploitation Les hyperviseurs de type 2, sont des logiciels qui s'exécutent sur un système d'exploitation hôte, comme VirtualBox ou VMware Workstation. Plus faciles à installer et à utiliser, ils sont souvent utilisés pour le développement, les tests ou l'apprentissage, mais ne sont pas adaptés aux environnements de production en raison de leur performance inférieure et de leur isolation moins robuste.

Limites des machines virtuelles

Malgré leurs avantages, les machines virtuelles présentent certaines limites :

- Consommation importante de ressources (chaque VM embarque un OS complet)
- Temps de démarrage relativement long
- Complexité de gestion à grande échelle

Ces limites ont conduit à l'émergence des conteneurs, plus légers et plus rapides.

Types de virtualisation

Lien avec le cloud

La virtualisation est la base du cloud computing. Elle permet aux fournisseurs comme AWS de mutualiser leurs infrastructures et de proposer des ressources à la demande.

Par exemple, une instance EC2 correspond à une machine virtuelle exécutée sur un serveur physique partagé avec d'autres utilisateurs.

Cela permet :

- une facturation à l'usage
- une scalabilité rapide
- une isolation entre clients

1.3 Différentes méthodes

Cette section présente les trois principaux modèles de déploiement cloud : **public**, **privé** et **hybride**.

Chaque modèle répond à des besoins spécifiques en termes de flexibilité, de sécurité, de coût et de souveraineté.

1.3.1 Cloud Public

Définition

Le **cloud public** est un modèle où les ressources informatiques (serveurs, stockage, applications) sont mises à disposition par un fournisseur tiers via Internet.

Ces ressources sont partagées entre plusieurs clients (multi-tenancy) et accessibles à la demande, avec une facturation généralement basée sur l'usage (pay-as-you-go). On le trouve chez des fournisseurs comme AWS, Microsoft Azure ou Google Cloud Platform.

Avantages

Ce modèle permet aux entreprises de bénéficier d'une infrastructure évolutive sans avoir à investir dans du matériel coûteux ou à gérer la maintenance. Les fournisseurs de cloud public offrent une large gamme de services (calcul, stockage, bases de données, intelligence artificielle) qui peuvent être rapidement provisionnés et intégrés. Il permet également une grande flexibilité, avec la possibilité d'ajuster les ressources en fonction des besoins, et une accessibilité mondiale grâce à des centres de données répartis géographiquement. C'est le fournisseur qui gère la sécurité physique, les mises à jour et la disponibilité, ce qui réduit la charge de travail pour les utilisateurs finaux.

Inconvénients

Mais ce type de modèle présente aussi des inconvénients, notamment une dépendance au fournisseur, qui peut entraîner une forte contrainte vis-à-vis de ses conditions tarifaires. Mais aussi des enjeux de sécurité et de conformité, puisque les données sont hébergées chez un tiers, ce qui peut soulever des questions réglementaires (par exemple vis-à-vis du RGPD). La latence peut également être un problème, surtout si les utilisateurs sont éloignés des centres de données du fournisseur, bien que la plupart des grands fournisseurs offrent des options pour choisir la région de déploiement. Enfin, le manque de contrôle sur l'infrastructure sous-jacente peut être un frein pour certaines organisations,

1.3.2 Cloud Privé

Définition

Le **cloud privé** est une infrastructure cloud dédiée à une seule organisation.

Il peut être hébergé en interne (on-premise) ou chez un prestataire externe, mais reste exclusif à l'organisation. Dans le cas d'un cloud privé on-premise, l'organisation possède et gère elle-même l'infrastructure, offrant un contrôle total sur les ressources et les solutions de sécurité.

j'ai encore pas mal de lecture la dessus, je comprends pas tout, je finira de rédiger cette partie plus tard

AWS VPC ?

<https://aws.amazon.com/fr/vpc/> eux, expliquent un peu l'intérêt du contrôle "total". remarque aussi que le moindre EC2 est placé dans un VPC par défaut c'est quoi le contrôle total ?

Exemples de solutions

- OpenStack (open source)
- VMware Cloud
- Microsoft Azure Stack
- Clouds souverains (ex : OVHcloud, 3DS Outscale)

Ce qui suit n'est pas rédigé .. c'est du copier coller chatgpt => PAS BON!

Avantages

- **Sécurité renforcée** : Isolation des données et contrôle total sur l'infrastructure.
- **Conformité** : Idéal pour les secteurs réglementés (santé, finance, défense).
- **Personnalisation** : Adaptation fine des ressources aux besoins spécifiques.
- **Performances** : Latence réduite et ressources dédiées.

Inconvénients

- **Coût élevé** : Investissement initial important en matériel et maintenance.
- **Complexité de gestion** : Nécessite des compétences internes pour l'administration.
- **Scalabilité limitée** : L'extension des ressources peut être lente et coûteuse.
- **Risque de sous-utilisation** : Les ressources dédiées peuvent être sous-exploitées, entraînant un gaspillage.
- **Manque de redondance** : En cas de panne, la récupération peut être plus complexe sans les infrastructures de secours d'un cloud public.

Cas d'usage typiques

- Environnements sensibles (données médicales, financières).
- Applications critiques nécessitant une haute disponibilité.
- Projets nécessitant une souveraineté totale sur les données.

1.3.3 Cloud Hybride

Définition

Le **cloud hybride** combine les modèles public et privé, permettant aux organisations de bénéficier des avantages des deux approches.

Les ressources sont orchestrées pour communiquer entre elles, offrant flexibilité et optimisation des coûts.

Exemples d'architectures

- Utilisation d'un cloud privé pour les données sensibles et d'un cloud public pour les pics de charge.
- Intégration d'AWS Outposts (extension d'AWS sur site) avec un cloud privé OpenStack.

Avantages

- **Flexibilité** : Adaptation dynamique des ressources selon les besoins.
- **Optimisation des coûts** : Utilisation du cloud public pour les charges variables et du privé pour les données critiques.
- **Continuité d'activité** : Redondance entre clouds pour une haute disponibilité.
- **Conformité** : Possibilité de garder les données sensibles en local tout en utilisant le cloud public pour le reste.

Inconvénients

- **Complexité** : Gestion et intégration des environnements public/privé.
- **Sécurité** : Nécessite une stratégie unifiée pour protéger les échanges entre clouds.
- **Coût de mise en place** : Investissement initial pour l'interconnexion et l'orchestration.

Cas d'usage typiques

- Entreprises avec des besoins variables (ex : e-commerce avec pics saisonniers).
- Projets nécessitant à la fois souveraineté et scalabilité (ex : santé, administration).
- Migration progressive vers le cloud (stratégie "lift-and-shift").

on ne ressent pas une grande profondeur de connaissance : illustre , explique, donne du corps!!!

1.4 Modèles d'exploitation

IaaS PaaS SaaS

2. Objectif du travail

2.1 Qu'est ce que le cloud

Dans cette section, nous allons essayer de définir ce qu'est le **cloud computing**, ses caractéristiques, et les différentes formes qu'il peut prendre.

Il est important de comprendre les bases du cloud computing pour pouvoir analyser les besoins et faire des choix éclairés pour notre projet.

Il faut noter que le cloud computing est un domaine en constante évolution, et il existe de nombreuses définitions et interprétations différentes. Cependant, nous allons essayer de donner une définition générale qui englobe les principales caractéristiques du cloud à l'heure actuelle.

Le cloud computing est un **modèle de fourniture de services informatiques** qui permet d'accéder à des ressources informatiques (serveurs, stockage, bases de données, réseaux, logiciels, etc.) via Internet.

Contrairement à l'informatique "traditionnelle", où les ressources sont hébergées localement, le cloud externalise ces infrastructures vers des **centres de données distants**, gérés par des prestataires spécialisés. [2, 6].

Contrairement à l'informatique "traditionnelle", où les ressources sont hébergées localement, le paradigme du cloud externalise ces infrastructures vers des **centres de données distants**, souvent gérés par des prestataires spécialisés. [2, 6].

sinon ca te met en porte à faux parce que toi tu veux faire un cloud qui n 'externalise pas complètement) même relation qu'entre Internet et Intranet

2.1.1 Caractéristiques clés (NIST/ISO, 2026)

Ce qui suit n'est pas rédigé .. c'est du copier coller chatgpt => PAS BON!

- **Accès réseau omniprésent** : Les ressources sont accessibles depuis n'importe quel appareil connecté à Internet.

- **Pool de ressources mutualisées** : Les infrastructures sont partagées et allouées dynamiquement selon les besoins (virtualisation, conteneurs).
- **Élasticité et scalabilité** : Les ressources peuvent être ajustées automatiquement, en temps réel, pour répondre à la demande.
- **Self-service à la demande** : L'utilisateur peut provisionner des ressources (ex : machines virtuelles, bases de données) sans intervention humaine du fournisseur.
- **Facturation à l'usage** : Modèle économique "pay-as-you-go", où le coût est proportionnel à la consommation réelle.

2.1.2 Évolutions récentes (2026)

Ce qui suit n'est pas rédigé .. c'est du copier coller chatgpt => PAS BON!

En 2026, le cloud n'est plus seulement un **lieu de stockage** ou d'hébergement, mais une **plateforme d'innovation** intégrant :

- **L'intelligence artificielle** (ex : GPU-as-a-Service pour l'entraînement de modèles, AIOps pour l'optimisation automatique des ressources) [4].
- **L'Edge Computing** : traitement des données à la périphérie du réseau pour réduire la latence, notamment pour l'IoT et les applications temps réel [7].

Plus que ça!!! regarde le concept de continuum computing

- Les **architectures hybrides et multi-cloud** : les entreprises combinent clouds publics, privés et infrastructures locales pour éviter le *vendor lock-in* et optimiser flexibilité/sécurité [1].

vendor lock-in ?

- La **souveraineté des données** : développement de clouds locaux (ex : OVHcloud en France) pour se conformer aux réglementations comme le RGPD [3].

2.1.3 Pourquoi cette définition est-elle importante pour notre projet ?

Ce qui suit n'est pas rédigé .. c'est du copier coller chatgpt => PAS BON!

rédiger ce n'est pas qu'énoncer ...c'est expliquer

Cette compréhension permet d'identifier :

- Les **avantages** du cloud (flexibilité, coût, innovation) mais aussi ses **limites** (dépendance, sécurité, latence).

tu es sûr que le cout est un avantage ?

- Les **critères de choix** entre cloud public, privé, hybride ou *self-hosted*, en fonction des besoins en souveraineté, performance et budget.

public ? privé ? on ne sait pas encore ce que c'est !

2.2 Contexte d'utilisation

Dans le cadre du projet **DS4H**, il m'a été confié la mission de concevoir et déployer une **solution self-hosted** pour remplacer le service **Render** utilisé dans le cours d'**IoT**.

L'objectif est de fournir une alternative locale, souveraine et maîtrisée, permettant aux étudiants et enseignants de déployer, gérer et monitorer des applications IoT sans dépendre d'un service cloud externe.

quel modèle ? IaaS, PaaS ?

Cette initiative s'inscrit dans une démarche visant à :

- **Réduire la dépendance** aux services tiers, en alignement avec les principes de souveraineté numérique et de résilience pédagogique.
- **Reproduire les fonctionnalités essentielles** de Render (déploiement continu, gestion des environnements, monitoring basique) tout en s'adaptant aux besoins spécifiques du cours (protocoles IoT, gestion des devices, etc.).
- **Faciliter l'apprentissage** en offrant un environnement contrôlé et accessible, propice à l'expérimentation des concepts d'IoT et d'infrastructure.

L'absence de contraintes techniques précises laisse une grande flexibilité pour le choix de la solution, qui sera déterminé lors de l'**analyse des besoins** et des choix technologiques présentés dans les sections suivantes.

2.3 Analyse des besoins

Le but de ce projet étant d'implémenter une solution de cloud self-hosted pour remplacer Render, il est essentiel de comprendre les besoins spécifiques liés à ce contexte d'utilisation.

- **Deploiement facile** : Les utilisateurs doivent pouvoir déployer leurs code sans que la complexité technique ne soit un obstacle.
- **Gestion des comptes utilisateurs** : Il doit être possible de créer et gérer des comptes pour les étudiants.
- **Monitoring basique** : Les utilisateurs doivent pouvoir surveiller l'état de leurs applications (logs, ressources utilisées).
- **Coût maîtrisé** : La solution doit être économique à mettre en place et à maintenir, en évitant les coûts récurrents élevés.

3. Choix de la techno

Face aux besoins identifiés — déploiement simple, gestion d'utilisateurs, monitoring basique et coût maîtrisé — plusieurs solutions de cloud self-hosted ont été évaluées. L'objectif n'est pas de construire une infrastructure de production industrielle, mais de fournir un environnement pédagogique fonctionnel, maintenable par une seule personne et accessible à des étudiants sans expertise DevOps.

3.1 Solutions envisagées

3.1.1 OpenStack

OpenStack est la référence du cloud privé open-source. Il reproduit fidèlement les primitives d'un cloud public (instances de calcul, réseaux virtuels, stockage objet, etc.) et est utilisé par de nombreux opérateurs en production.

Cependant, sa complexité d'installation et d'administration est incompatible avec notre contexte : il nécessite typiquement plusieurs machines physiques dédiées, une équipe pour le maintenir, et une courbe d'apprentissage très significative. Ce serait, pour reprendre l'analogie du chapitre précédent, utiliser une infrastructure de datacenter pour remplacer un service de déploiement étudiant. OpenStack a donc été écarté dès le début de l'évaluation.

3.1.2 Kubernetes (k3s)

Kubernetes est le standard de facto pour l'orchestration de conteneurs à grande échelle. Sa version allégée, k3s, réduit les prérequis matériels tout en conservant l'essentiel de l'API Kubernetes.

Le problème est similaire à OpenStack : Kubernetes est conçu pour gérer des centaines de services sur des dizaines de nœuds, avec une gestion fine des ressources, de la haute disponibilité et du réseau. Pour notre cas d'usage, cela représente une complexité

accidentelle¹ importante. Par ailleurs, Kubernetes n'intègre pas nativement d'interface utilisateur pour les déploiements applicatifs ni de gestion de comptes étudiants.

3.1.3 Dokku

Dokku se présente comme un Heroku self-hosted minimaliste. Il permet de déployer une application via un simple `git push`, ce qui correspond bien au niveau de simplicité recherché.

Toutefois, Dokku reste un outil orienté ligne de commande, sans interface web native pour les utilisateurs finaux. La gestion multi-utilisateurs y est limitée et repose essentiellement sur des accès SSH. Pour des étudiants habitués à une interface web comme Render, la transition aurait été moins naturelle.

3.1.4 CapRover

CapRover propose une interface web et un déploiement par conteneurs Docker, avec une approche plus accessible que Kubernetes. Il supporte le déploiement depuis un dépôt Git et propose une gestion basique des applications.

Cependant, la gestion des utilisateurs y est rudimentaire : CapRover fonctionne historiquement avec un unique compte administrateur, et bien que des évolutions aient été apportées, la séparation des droits entre projets reste peu granulaire. C'est un critère important pour notre contexte, où chaque étudiant doit accéder uniquement à ses propres applications.

3.1.5 Coolify

Coolify est une solution récente et activement développée qui propose une interface moderne, le déploiement depuis Git, la gestion de bases de données, et un système multi-utilisateurs. Elle s'est révélée être un concurrent sérieux à Dokploy sur le papier.

La raison de son élimination est cependant pragmatique : l'image Docker de Coolify dépasse les 20 Go une fois déployée avec ses dépendances. Sur le VPS utilisé pour ce projet, dont l'espace disque est limité, cela rendait l'installation tout simplement impossible sans sacrifier la majeure partie des ressources disponibles pour les applications étudiantes elles-mêmes.

C'est un exemple concret de contrainte d'infrastructure qui ne ressort pas des comparatifs en ligne, mais qui s'impose dès qu'on passe à la pratique.

1. On parle de complexité accidentelle lorsque la complexité vient de l'outil choisi et non du problème à résoudre.

3.2 Tableau comparatif

Solution	Interface web	Multi-utilisateurs	Déploiement Git	Complexité	Adé
OpenStack	✓	✓	~	Très élevée	
Kubernetes (k3s)	~	~	~	Élevée	
Dokku	×	Limitée	✓	Faible	
CapRover	✓	Limitée	✓	Moyenne	
Coolify	✓	✓	✓	Faible	
Dokploy	✓	✓	✓	Faible	

TABLE 3.1 – Comparaison des solutions de cloud self-hosted évaluées

3.3 Justification du choix : Dokploy

Au terme de cette évaluation, Dokploy a été retenu pour plusieurs raisons convergentes.

D’abord, il répond directement aux quatre besoins identifiés : déploiement depuis Git en quelques clics, gestion de comptes utilisateurs avec des permissions par projet, monitoring des logs et des ressources depuis le dashboard, et installation en une seule commande sur un serveur Linux standard.

Ensuite, son niveau de complexité est cohérent avec les ressources disponibles pour ce projet : une seule machine, un seul administrateur, et des utilisateurs qui ne doivent pas avoir à se préoccuper de l’infrastructure sous-jacente.

Enfin, contrairement à Kubernetes ou OpenStack, Dokploy ne cherche pas à être un cloud généraliste. C’est précisément ce qui en fait un choix pertinent ici : il résout notre problème sans introduire une complexité accidentelle disproportionnée.

4. Mise en place de l'infrastructure

4.1 Dokploy

Dokploy est une solution open-source de déploiement de service et base de donnée. Elle permet de déployer facilement des applications à partir d'un dépôt Git, en utilisant des conteneurs Docker.

4.1.1 Installation et configuration

L'installation de Dokploy est relativement simple et peut être réalisée en une seule commande sur un serveur Linux. Une fois installé, il faut créer des identifiants administrateur et configurer les paramètres de base (certificats SSL, domaine).

4.1.2 Services proposés par Dokploy

Dokploy offre plusieurs services clés :

- **Déploiement continu** : Intégration avec Docker et les plateformes Git (GitHub, GitLab, Gitea, Bitbucket, etc.) pour déployer automatiquement les applications à chaque push, avec la possibilité de déposer directement les fichiers.
- **Gestion des bases de données** : Dokploy propose une interface pour créer et gérer des bases de données, avec des options de sauvegarde et de restauration.
- **Monitoring** : Visualisation des logs et des ressources utilisées par les applications.
- **Interface utilisateur** : Dashboard web pour gérer les applications et les utilisateurs, l'administrateur peut créer des comptes et les associer à différents projets en paramétrant les permissions (création d'application, accès aux logs, etc...). Et les utilisateurs peuvent se connecter pour déployer et monitorer leurs applications en gérant tout depuis le panel.

Pour les administrateurs, Dokploy offre des fonctionnalités de gestion avancées, telles que la configuration des domaines personnalisés, la gestion des certificats SSL, la possibilité de définir des règles de déploiement spécifiques pour chaque projet. Il permet également de configurer plusieurs machines sur lesquelles déployer les applications, offrant ainsi une certaine flexibilité dans la gestion de l'infrastructure. Consultable facilement depuis le

dashboard, les administrateurs peuvent suivre l'état de chaque machine, les applications déployées et les ressources utilisées.

4.1.3 Avantages et inconvénients de Dokploy

Avantages

Il est facile à installer et à utiliser, avec une interface conviviale. Il offre une bonne intégration avec Git et supporte plusieurs types de bases de données. Cela permet de répondre rapidement aux besoins.

Inconvénients

Il peut être limité en termes de scalabilité et de personnalisation. La communauté est relativement petite, ce qui peut poser des problèmes de support et de développement à long terme. Bien qu'il offre la possibilité d'avoir plusieurs machines sur lesquelles déployer, il n'est pas conçu pour gérer une infrastructure complexe ou à grande échelle.

4.2 Cas d'usage : déploiement d'un RNN de classification de texte

Afin d'illustrer les capacités et les limites de notre solution de cloud self-hosted, nous avons choisi de déployer un modèle de réseau de neurones récurrent (RNN) pour la classification de texte. Nous disposons de 3 fichiers :

- **RNN_factory.py** : genere le RNN et l'entraîne
- **RNN_client.py** : client qui envoie des requetes de classification
- **RNN_server.py** : serveur qui reçoit les requetes et retourne les résultats

L'objectif est de déployer le serveur et de simuler des requetes afin de tester la scalabilité et les performances de notre infrastructure.

4.2.1 Déploiement avec Dokploy

Dans un premier temps, un nouveau projet est créé au sein de Dokploy sous l'intitulé "RNN-Classification". À l'intérieur de ce projet, une application est ensuite ajoutée et nommée "RNN-Server".

Plusieurs approches de déploiement sont envisageables. Il est possible d'utiliser une image Docker préalablement construite et publiée sur un registre (tel que Docker Hub ou GitHub Container Registry), que Dokploy peut déployer directement. Une autre option

consiste à fournir directement les fichiers sources à Dokploy afin qu'il construise l'image à partir d'un Dockerfile. Enfin, il est également possible de connecter un dépôt Git (GitHub, GitLab, etc.) afin d'automatiser le processus de déploiement lors de chaque mise à jour du code.

Dans le cadre de ce projet, la troisième approche a été retenue afin de bénéficier d'un flux de travail plus intégré et automatisé. Un dépôt Git est donc créé pour le projet, dans lequel sont ajoutés les fichiers `RNN_factory.py`, `RNN_client.py` et `RNN_server.py`.

Un fichier `Dockerfile` est ensuite défini afin de spécifier les étapes de construction de l'image de l'application, ainsi qu'un fichier `requirements.txt` listant les dépendances nécessaires, telles que TensorFlow et Flask.

Du côté de Dokploy, le projet est configuré pour se connecter au dépôt Git et pour utiliser le Dockerfile afin de construire automatiquement l'image à chaque nouvelle version poussée sur le dépôt. La commande de lancement du serveur est également définie, à savoir : `gunicorn -bind 0.0.0.0:5000 RNN_server:app`.

Enfin il faut paramétrer le domaine sur lequel va répondre l'api, ici j'ai choisi `rnn.dokpoly.claveil.fr` et le port (ici : 5000).

Une fois la configuration terminée, le déploiement peut être initié. Dokploy se charge alors de construire l'image Docker, de la déployer sur le cluster, puis d'exposer le service via une URL accessible.

4.2.2 Tests de scalabilité et de performance

Une fois le serveur déployé, on utilise le script `RNN_client.py` pour envoyer des requêtes de classification afin de tester les performances de l'application.

On peut remarquer qu'en faisant des requêtes en continu l'application utilise déjà 40% du CPU qui lui est alloué. Cela illustre les limites de notre infrastructure : avec une seule instance du serveur, la capacité de traitement est rapidement saturée.

4.3 Scalabilité avec Dokploy

Nous allons donc tester la scalabilité de notre solution en augmentant le nombre d'instances du serveur. Dokploy permet de faire cela facilement depuis le dashboard, en ajustant le nombre de conteneurs déployés pour l'application.

Pour cela, Dokploy nécessite un **Docker Registry**, qui est un service de stockage et de distribution d'images Docker. Il joue le rôle d'un dépôt centralisé depuis lequel chaque instance peut récupérer l'image de l'application à déployer. Sans ce registry, les différents conteneurs ne pourraient pas accéder à une version commune et cohérente de l'image.

Nous utilisons ici le **GitHub Container Registry (GHCR)**, qui s'intègre nativement avec notre dépôt GitHub et ne nécessite pas d'infrastructure supplémentaire. Une fois le registry configuré dans Dokploy, il suffit d'ajuster le nombre de réplicas depuis l'onglet **Advanced** de l'application. Traefik, le reverse proxy intégré à Dokploy, se charge alors automatiquement de répartir le trafic entre les différentes instances.

Bibliographie

- [1] Objet Connecté. Cloud computing en 2026 : tendances, innovations et défis. 2026.
- [2] International Organization for Standardization. Cloud computing — overview and vocabulary (iso/iec 17788 :2024), 2026.
- [3] Tech Insider. Souveraineté numérique france 2026 : Cloud souverain. 2026.
- [4] LeBigData.fr. Cloud computing – définition, avantages et exemples d'utilisation. 2026.
- [5] Computer History Museum. George stibitz - pioneer of remote computing, 2012.
- [6] National Institute of Standards and Technology. Nist definition of cloud computing, 2026.
- [7] Syslearn. Cloud computing : Caractéristiques essentielles et avantages stratégiques. 2026.
- [8] Wikipedia. Adder (electronics). 2026.